

Interrupts

Imagine that you're in the kitchen whipping up a batch of chocolate chip cookies. You're following a recipe, which is just a program for cooks. You're the only one home, so you need to know if someone comes to the door. We'll represent your activity using a *flowchart*, which is a type of diagram used to express how things work, as shown in Figure 5-6.

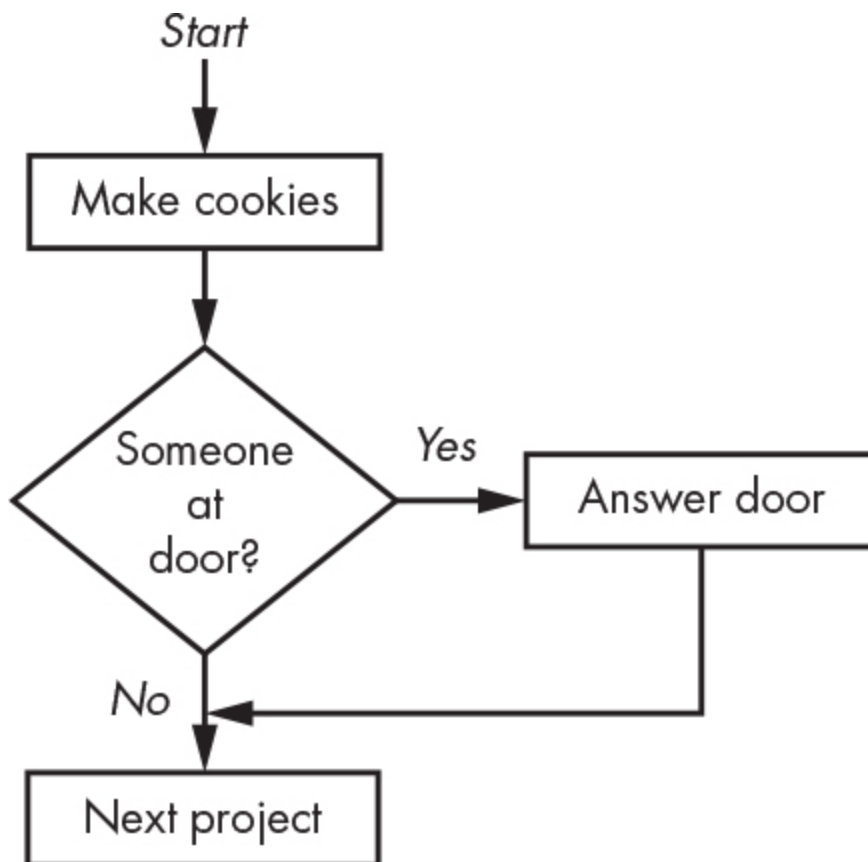


Figure 5-6: Home alone making cookies #1

This might work if someone really patient comes to the door. But let's say that a package is being delivered that needs your signature. The delivery person isn't going to wait 45 minutes, unless they can smell the cookies and are hoping to get some. Let's try something different, like Figure 5-7.

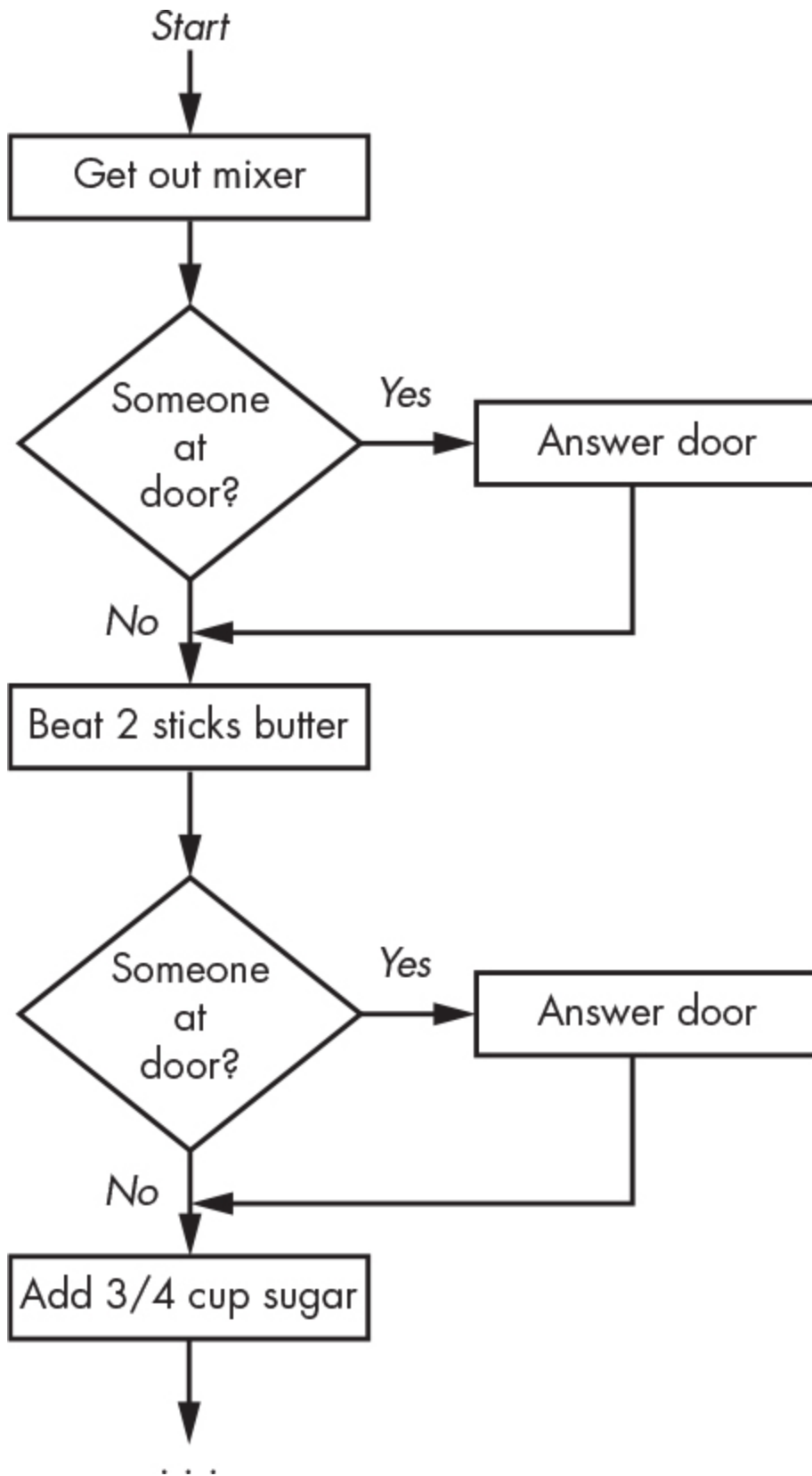


Figure 5-7: Home alone making cookies #2

This technique is called *polling*. It works, but not very well. You're less likely to miss your delivery, but you're spending a lot of time checking the door.

We could divide up each of the cookie-making tasks into smaller subtasks and check the door in between them. That would improve your chances of receiving the delivery, but at some point you'd be spending more time checking the door than making cookies.

This is a common and important problem for which there is really no software solution. It's not possible to make this work well by rearranging the structure of a program. What's needed is some way to *interrupt* a running program so that it can respond to something external that needs attention. It's time to add some hardware features to the execution unit.

Pretty much every processor made today includes an *interrupt* unit. Usually it has pins or electrical connections that generate an interrupt when wiggled appropriately. *Pin* is a colloquial term for an electrical connection to a chip. Chips used to have parts that looked like pins, but as devices and tools have gotten smaller, many other variants have emerged. Many processor chips, especially microcomputers, have *integrated peripherals* (on-chip I/O devices) that are connected to the interrupt system internally.

Here's how it works. A peripheral needing attention generates an *interrupt request*. The processor (usually) finishes up with the currently executing instruction. It then puts the currently executing program on hold and veers off to execute a completely different program called an *interrupt handler*. The interrupt handler does whatever it needs to do, and the main program continues from where it left off. Interrupt handlers are functions.

The equivalent mechanism for the cookie project is a doorbell. You can happily make cookies until you're interrupted by the doorbell, although it can be annoying to be interrupted by pollsters. There are a few things to consider. First is your *response time* to the interrupt. If you spend a long time gabbing with the delivery person, your cookies may

burn; you need to make sure that you can service interrupts in a timely manner. Second, you need some way to save your *state* when responding to an interrupt so that you can go back to whatever you were doing after *servicing* it. For example, if the interrupted program had something in a register, the interrupt handler must save the contents of that register if it needs to use it and then restore it before returning to the main program.

The interrupt system uses a stack to save the place in the interrupted program. It is the interrupt handler's job to save anything that it might need to use. This way, the handler can save the absolute minimum necessary so that it works fast.

How does the computer know where to find the interrupt handler? Usually, there's a set of reserved memory addresses for interrupt vectors, one for each supported interrupt. An *interrupt vector* is just a pointer, the address of a memory location. It's similar to a vector in math or physics—an arrow that says, “Go there from here.” When an interrupt occurs, the computer looks up that address and transfers control there.

Many machines include interrupt vectors for exceptions including stack overflow and using an invalid address such as one beyond the bounds of physical memory. Diverting exceptions to an interrupt handler often allows the interrupt handler to fix problems so that the program can continue running.

Typically, there are all sorts of other special interrupt controls, such as ways to turn specific interrupts on and off. There is often a *mask* so that you can say things like “hold my interrupts while the oven door is open.” On machines with multiple interrupts, there is often some sort of *priority* ordering so that the most important things get handled first. That means that the handlers for lower-priority interrupts may themselves be interrupted. Most machines have one or more built-in *timers* that can be configured to generate interrupts.

Operating systems, discussed in the next section, often keep access to the *physical* (hardware) interrupts out of reach from most programs. They substitute some sort of *virtual* or software interrupt system. For

example, the UNIX operating system has a *signal* mechanism. More recently developed systems call these *events*.